脆弱性候補の影響有無の判定とその管理の効率化に向けた基礎検討

小津 喬[1,a)]　山口 真司[1,b)]　横山 浩之[1,c)]

概要：サプライチェーンセキュリティの向上のため，ソフトウェア部品表（Software Bill of Materials, SBOM）を用いた脆弱性管理が普及している．しかし，パッケージ管理情報を取得できない状況では，既存のツールはソフトウェアの開発元が公開したバージョンに基づいて構成要素を同定し，セキュリティアップデートとしてバックポートされた修正等，配布版固有の修正の適用状況を認識できない場合がある．このため，修正をバックポートした配布版であっても脆弱と判定される場合があり，過剰な候補が製品の検収及び保守・運用における精査の負担とアラート疲労を招く．本稿は，脆弱性候補の影響有無を判定して脆弱性管理を効率化する構想の基礎検討として，Ubuntu を対象に以下の 4 点を提示する．第一に，配布版固有情報の欠落が構成要素の同定結果と候補集合へ伝播する過程を整理する．第二に，Canonical が公開する脆弱性情報を主要根拠とし，バイナリ上の修正適用状況を補助根拠として，VEX（Vulnerability Exploitability eXchange）状態の判定基準と判定停止条件を定義する．第三に，状態を確定できない候補の保留理由と後続調査を対応付ける管理モデルを提示する．第四に，本枠組みの有効性を検証するための比較条件及び評価指標を示す．すべての候補を影響ありとする状況と比較し，状態と根拠を区別して提示することでアラートの緩和を図る．

キーワード：サプライチェーンセキュリティ，脆弱性管理，SBOM，VEX，Ubuntu

A Preliminary Study toward Determining Whether Vulnerability Candidates Affect Products and Improving the Efficiency of Vulnerability Management

Takashi Ozu[1,a)]　Shinji Yamaguchi[1,b)]　Hiroyuki Yokoyama[1,c)]

Abstract: Vulnerability management using a Software Bill of Materials (SBOM) is spreading to improve supply chain security. However, when package management metadata is unavailable, existing tools identify components by the version published by the original software developer and may fail to recognize distribution-specific patches such as backported security fixes. A build with a backported fix may therefore be flagged as vulnerable, and excess candidates increase review burden and alert fatigue in product acceptance, maintenance, and operation. As a preliminary study toward judging candidate impact and improving vulnerability management, this paper presents four contributions for Ubuntu. First, we organize how the absence of distribution-specific metadata propagates to component identification and the candidate set. Second, using Canonical's vulnerability data as primary evidence and binary-level patch presence as supplementary evidence, we define criteria and halting conditions for assigning Vulnerability Exploitability eXchange (VEX) status. Third, we present a management model that pairs deferral reasons for undetermined candidates with subsequent investigation steps. Fourth, we describe comparison conditions and evaluation metrics for verifying the framework's effectiveness. Unlike treating all candidates as affected, presenting status and grounds separately aims to reduce alerts.

Keywords: Supply chain security, Vulnerability Management, SBOM, VEX, Ubuntu

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

ソフトウェアサプライチェーンの透明性を確保する手段として，ソフトウェア部品表（Software Bill of Materials, SBOM）の活用が進んでいる [1]．SBOM に記録された構成要素の識別情報と脆弱性データベースの照合により，脆弱性候補が列挙される．しかし，列挙された候補は調査開始点であり，対象製品が実際に影響を受けるとは限らない．候補の列挙と影響有無の確定は異なる工程であるが，両者が区別されずに扱われる場合がある．

本稿では，Ubuntu が配布するパッケージの基となるソフトウェアを開発・公開するプロジェクトを上流プロジェクトと呼び，その原版の版番号に由来するパッケージ版の部分を上流バージョンと呼ぶ．

Ubuntu を含む多くの Linux ディストリビューションでは，安定版の挙動変更を抑えるため，ソフトウェア全体を上流プロジェクトの新しいバージョンへ更新する代わりに，個別のセキュリティ修正を既存版へバックポートする運用が採られる [2]，[3]．この場合，修正済みのパッケージであっても上流バージョンの文字列は変化しないため，バージョンの照合のみでは修正状態を判別できない．パッケージ管理情報が利用可能であれば，配布版リビジョンを配布元の脆弱性情報及び変更履歴へ対応付けることで，修正の適用状況を追跡できるが，コンテナイメージやファームウェアの検収のように，パッケージ管理情報を欠く環境では追跡が困難となる．既存のツールは上流バージョンで構成要素を同定するため，配布版固有の修正適用状況を反映できない．バックポート修正を識別できなくなる一方，構成要素自体の同定失敗によって候補が欠落する場合もある．すなわち，パッケージ管理情報の欠落は候補数の増加と欠落の双方を生じさせ，候補集合の変化として扱う必要がある．過剰な候補は，製品の検収及び保守・運用における精査の負担とアラート疲労を招く．

本稿は，脆弱性候補の影響有無を判定して脆弱性管理を効率化する構想の基礎検討として，Ubuntu を対象に以下の 4 点を扱う．第一に，配布版固有情報の欠落が構成要素の同定結果と候補集合へ伝播する過程を整理する．第二に，VEX（Vulnerability Exploitability eXchange）状態の判定基準と判定停止条件を定義する．第三に，保留理由と後続調査を対応付ける管理モデルを提示する．第四に，将来の比較条件及び評価指標の設計を示す．

全体構成を図 1 に示す．

本稿の構成を示す．2 章では SBOM と脆弱性候補の関係，Ubuntu の版管理及び関連研究を整理する．3 章では問題設定と設計要件を述べる．4 章では影響有無の判定及び管理構想を説明する．5 章では評価計画を示す．6 章では考察及び制約を議論し，7 章でまとめと今後の課題を述べる．

[1] 株式会社国際電気通信基礎技術研究所（ATR）
Advanced Telecommunications Research Institute International (ATR)
[a)] ozu@atr.jp
[b)] shinji-yamaguchi@atr.jp
[c)] hr-yokoyama@atr.jp

2. 背景及び関連研究

2.1 SBOM と脆弱性候補

SBOM は，ソフトウェアに含まれる構成要素とその依存関係を機械可読な形式で記録した一覧である．2021 年の米国大統領令 14028 は，米国商務長官に SBOM の最小要素の公表を指示した [4]．日本においても経済産業省から導入手引 [1] が公表されている．代表的な SBOM 形式として SPDX と CycloneDX があり，構成要素の識別には CPE（Common Platform Enumeration）や PURL（Package URL）が用いられる．

構成要素の同定，SBOM の生成，脆弱性データベースとの照合，及び影響有無の判定は，それぞれ異なる処理段階である．同定の段階では，パッケージ管理データベースを参照する手法と，ファイルのメタデータやハッシュ値から推定する手法が存在する．前者は配布版固有の情報を取得できるが，パッケージ管理情報が利用できない環境では後者に頼らざるを得ない．照合の結果として列挙される候補は，対象製品が影響を受ける脆弱性ではなく，追加の調査を要する脆弱性候補である．候補から影響を受ける脆弱性を確定するには，配布版固有の修正状態や構成要素の対応関係を踏まえた判定が必要となる．

構成要素の同定における差は，候補集合の追加と欠落の双方を生じさせる．Stalnaker ら [5] は，SBOM の作成と利活用における 12 の課題を特定し，不正確な SBOM や検証の困難さを指摘している．特に，同一のソフトウェアであっても生成ツールや入力データの違いにより SBOM の内容が異なる点が課題として挙げられている．Kaur ら [6] は，科学データ解析用の 44 個のコンテナイメージを 4 種類のスキャナで分析し，スキャナ及び脆弱性データベースの組み合わせによる候補の差を報告しており，同一のイメージに対して検出される候補数がツールによって大きく異なることを示している．また，Anchore は Grype の事例を分析し，CPE に基づく照合及びパッケージ同定の誤りにより誤検知や検出漏れが生じること，並びにソフトウェアのエコシステムを考慮した脆弱性データによって照合結果を改善できることを報告している [7]．これらの知見は，同定と照合の各段階における差が候補集合へ伝播することを示唆している．

2.2 Ubuntu の版管理と配布元情報

Ubuntu では，パッケージ版が上流プロジェクトに由来するバージョン部分（以下，上流バージョン），Debian リビジョ

© 2026 Information Processing Society of Japan

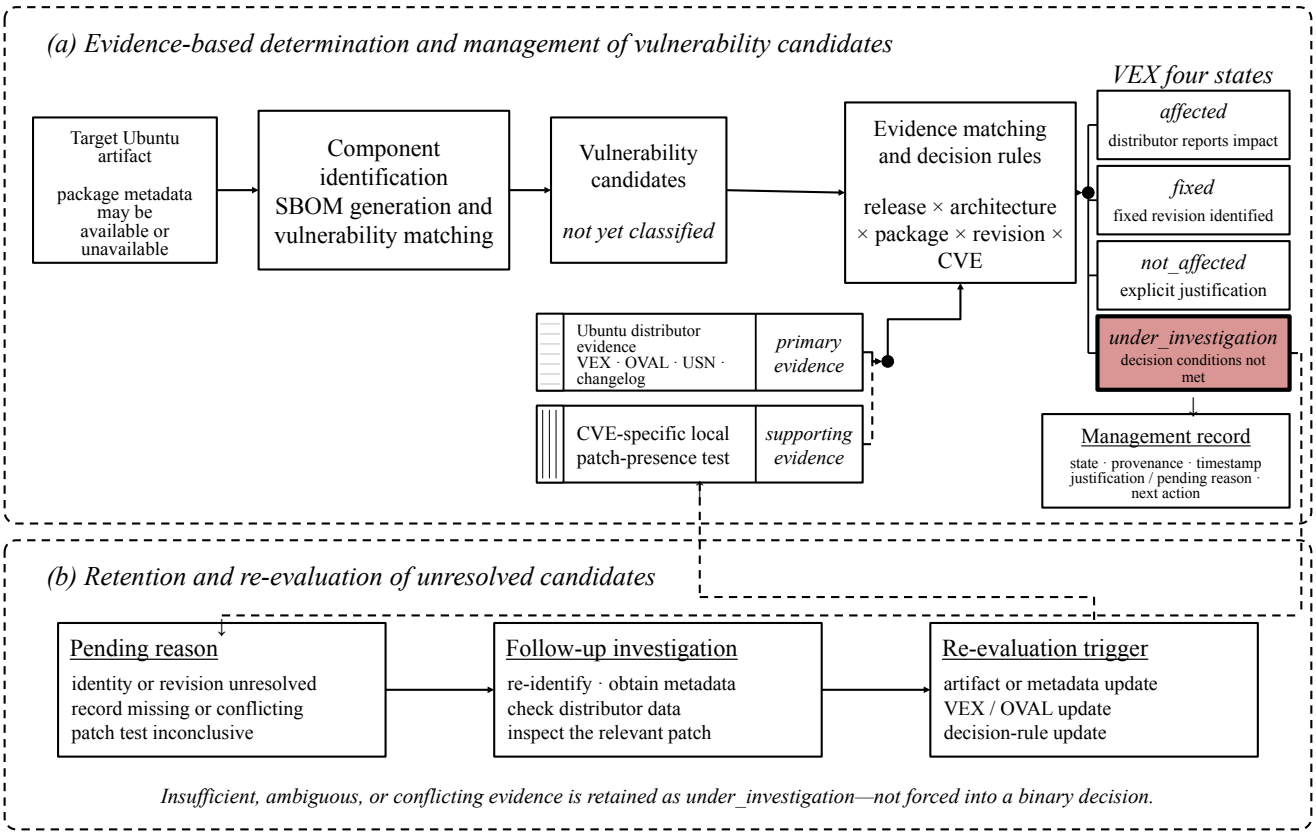


図 1: 脆弱性候補の影響判定及び保留・再評価の全体構成

ン及び Ubuntu リビジョンから構成される [8]．例えば，OpenSSL の場合，上流バージョンが `3.0.2` であるパッケージの完全な版文字列は `3.0.2-0ubuntu1.18` のように表記される．ここで `3.0.2` が上流バージョン，`0ubuntu1.18` が配布版リビジョンである．上流バージョンのセキュリティ修正をバックポートする場合，`3.0 (quilt)` 形式のソースパッケージでは，CVE（Common Vulnerabilities and Exposures）修正等のパッチが `debian/patches/` に格納される [9]．この場合，上流バージョンの `3.0.2` は変化せず，配布版リビジョンのみが `0ubuntu1.17` から `0ubuntu1.18` へ更新される [10]．したがって，上流バージョンに基づく照合のみでは修正済みか否かを判別できない．

さらに，Ubuntu ではひとつのソースパッケージから複数のバイナリパッケージが生成される．例えば，ソースパッケージ `openssl` からは `libssl3`，`openssl`，`libssl-dev` 等のバイナリパッケージが生成される．脆弱性データベースがソースパッケージ名で CVE を管理する一方，対象物上にはバイナリパッケージが存在するため，ソースパッケージとバイナリパッケージの対応関係を保持する必要がある．

また，dpkg 系パッケージ管理環境で APT のソースリポジトリが設定され，対象版のソースパッケージが提供されている場合には，`apt source` 又は `apt-get source` によりソースパッケージを取得できる [11]．取得物の `debian/changelog` には版ごとの変更と配布版リビジョンが記録される [8]．また，`3.0 (quilt)` 形式ではパッチ系列が `debian/patches/` に格納される [9]．したがって，上流バージョンだけから複数の配布版リビジョンが候補となる場合，取得可能な各候補の変更履歴及びパッチ系列を対象物で観測可能なコード等の特徴と照合することで，候補を絞り込める場合がある．一意な対応が得られた場合に限り，配布版リビジョンを同定できる．ただし，対象版のソースパッケージを取得できない場合又は候補間の差を対象物から観測できない場合には，配布版リビジョンを一意に同定できない．

Canonical は，サポート対象の Ubuntu リリースについて，CVE ごとの影響状態を VEX として公開している [12]．この VEX データは，リリースとソースパッケージの組に対して影響状態と修正版を記録しており，配布版リビジョンを含む版条件での照合が可能である．OVAL（Open Vulnerability and Assessment Language）による脆弱性評価データも提供されており [13]，OpenSCAP [14] は OVAL 定義に基づいてシステム上のパッケージ版を検査する評価手段として利用できる．USN（Ubuntu Security Notice）及び changelog も配布元情報として参照可能である．USN はセキュリティ修正の公開通知であり，対象パッケージと修正版の組を含む．changelog はパッケージの変更履歴であり，版ごとの変更と配布版リビジョンを記録しており，

© 2026 Information Processing Society of Japan

セキュリティ更新では CVE 修正が記載される場合がある．これらの情報は，脆弱性候補の影響有無を判定する根拠となり得る．

2.3 関連研究と本稿の位置付け

著者らは，Ubuntu のブートプロセスを bpftrace で動的に追跡し，指定シナリオで実行されないコンポーネントに起因する脆弱性検知の誤検知低減を報告した [15]．この研究は指定シナリオにおける利用観測に基づくフィルタリングであり，脆弱性の修正適用状況の確認や影響なしの判定を行うものではない．また，利用観測は特定のシナリオに依存するため，シナリオ外で発現する脆弱性を見落とす制約がある．本稿は，配布版固有の修正状態と根拠管理に焦点を当てる点で異なり，シナリオに依存しない判定を目指す．

Shourya ら [16] は，PSIRT（Product Security Incident Response Team）アナリストが脆弱性を効率的に評価するための手法として，候補推定と詳細評価の二段階化を提案した．第一段階で影響の可能性が高い候補を絞り込み，第二段階で詳細な評価を行うことで，アナリストの負荷を軽減する．本稿は，二段階化の考え方を共有しつつ，第一段階における判定基準として配布版固有の根拠を体系化し，第二段階への引き渡し条件を保留理由として明示する点が異なる．

Zhang ら [17] は，バイナリにおけるパッチの存在を検査する FIBER を提案した．FIBER は CVE 固有のソースレベルの修正特徴からバイナリ上のシグネチャを生成し，パッチ適用の有無を検査する．この手法は，ソースコードの差分が局所的であり，コンパイル後のバイナリにおいても識別可能な特徴が残る場合に有効である．本稿では，この種のバイナリパッチ存在検査を補助根拠として位置付ける．ただし，関数のインライン化，対象コードの独自改変及びバックポート時のパッチ調整等により，偽陰性が生じる場合がある [17]．この制約を踏まえ，バイナリ検査を単独の判定根拠とはせず，配布元情報による判定の整合確認に用いる．

Deng ら [18] は，上流脆弱性の PoC（Proof of Concept）を基に，ファイル入力を受け取る下流ソフトウェアで当該脆弱性の悪用可能性を検証する ChainFuzz を提案した．この研究は候補列挙と下流での実影響確認の差を扱う点で本稿と問題意識を共有するが，上流プロジェクトの PoC に依存し，評価対象もファイル入力を受け取る下流ソフトウェアに限られる．本稿は PoC を前提とせず，配布元情報を主要根拠とする点で手法が異なる．

表 1 に，関連研究と本稿の比較を示す．

表 1: 関連研究との比較

手法	主要根拠	動的解析	保留管理
著者ら [15]	利用観測	要	—
Shourya [16]	スコア	不要	—
FIBER [17]	バイナリ	不要	—
ChainFuzz [18]	PoC	要	—
本稿	配布元情報	不要	有

3. 問題設定と設計要件

3.1 想定利用場面と対象範囲

本稿が想定する利用場面は，納入物の検収，保守，脆弱性対応及び再評価である．検収では，納入されたコンテナイメージやファームウェアに含まれる脆弱性候補を評価し，受入可否の判断材料とする．保守では，新たに公開された CVE に対して既存の判定結果を再評価し，対応の要否を判断する．脆弱性対応では，影響を受ける候補に対して修正の適用計画を策定する．再評価では，パッケージの更新や配布元情報の追加を契機として，保留中の候補の状態を更新する．

Ubuntu 公式アーカイブ由来のパッケージを中核対象とし，第三者リポジトリ，独自ビルド及び他ディストリビューションは対象外とする．この範囲限定は，Canonical が公開する配布元情報を主要根拠とするためである．

3.2 判定単位と入出力

判定単位を「Ubuntu リリース × アーキテクチャ × パッケージ識別子 × 配布版リビジョン × CVE」とする．リリースを含めるのは，同一パッケージであっても Ubuntu リリースごとに適用されるパッチが異なるためである．アーキテクチャを含めるのは，バイナリレベルの検査結果がアーキテクチャに依存するためである．配布版リビジョンを含めるのは，前述の通りバックポート修正の有無を判別する鍵となるためである．

主な入力は対象物と Canonical の配布元情報（VEX，OVAL，USN，changelog）である．SBOM 及び候補一覧は，本手法の処理フロー内で対象物から生成される中間生成物として位置付ける．補助入力として，利用可能な場合にバイナリパッチ検査結果を受け付ける．

出力は，VEX 状態，根拠，出典，取得時点，保留理由及び次の調査工程である．出力には判定時点の入力データの版を含め，後日の再評価において判定の再現と差分の特定を可能とする．

3.3 設計要件

判定枠組みの設計要件を以下に示す．

(1) 候補と状態を分離する．候補の列挙と影響有無の判定

© 2026 Information Processing Society of Japan

を独立した工程として扱う．
(2) 状態と根拠を分離する．付与された VEX 状態が何に基づくかを明示する．
(3) 根拠不足を二値判定へ強制変換しない．情報が不十分な候補は `under_investigation` として保留する．
(4) 配布元の見解と本手法による派生判定を区別する．Canonical の公式見解と，それを基に本手法が導いた判定の出自を分離する．
(5) 情報更新時に再評価できる来歴を保存する．判定時点の入力データ，適用した規則及び出力を記録し，情報源の更新時に差分再評価を可能とする．

4. 影響有無の判定及び管理構想

4.1 全体構成

本手法は，対象物取得，構成要素同定，候補列挙，Canonical の配布元情報との対応付け，バイナリ確認及び VEX 状態出力の処理フローで構成される．各段階は独立して実行可能であり，前段の出力を後段の入力として受け渡す．全体構成は図 1 に示した通りであり，以下では各段階を説明する．

対象物として，コンテナイメージ，ファイルシステムの展開物又はインストール済みシステムを想定する．構成要素同定の段階では，パッケージ管理情報の利用可能性に応じた同定手順を適用し，SBOM を中間生成物として出力する．候補列挙の段階では，生成された SBOM と脆弱性データベースの照合により，候補一覧を中間生成物として出力する．配布元情報との対応付けの段階では，Canonical VEX，OVAL，USN 及び changelog を主な入力として参照し，各候補の影響状態を判定する．バイナリ確認の段階では，補助入力が利用可能な場合に CVE 固有のパッチ存在検査を実施する．VEX 状態出力の段階では，判定結果を VEX の 4 状態として出力する．

4.2 配布版情報を欠く構成要素の同定

パッケージ管理情報の利用可能性に応じて，構成要素の同定結果は以下の 4 段階に区分される．

(1) 配布版リビジョンまで一意に同定できる場合．パッケージ管理データベース（dpkg のステータスファイル等）が利用可能であり，上流バージョンと配布版リビジョンの組が一意に特定される．例えば，`openssl 3.0.2-0ubuntu1.18` のように完全な版文字列が取得できる．
(2) 複数の配布版リビジョンの候補が得られる場合．上流バージョンは特定できるが，配布版リビジョンを一意に決定できず，候補が複数存在する．例えば，上流バージョン `3.0.2` のパッケージがリポジトリに複数の配布版リビジョンで存在し，対象物がいずれに該当するかを特定できない場合である．候補版のソースパッケージを取得でき，候補間で異なる特徴を対象物から観測できる場合は，ソース及び付随情報との照合により候補を絞り込む．一意な対応が得られた場合は前項へ移行し，得られない場合は候補集合を保持する．
(3) 上流バージョンのみが同定できる場合．パッケージ管理情報を欠き，ELF バイナリのバージョン文字列やライブラリの SONAME 等のファイル解析により上流バージョンを推定する．Ubuntu リリース又はパッケージとの対応を確定できず，配布版リビジョンの候補集合を信頼できる形で列挙できない．このため，バックポート修正の有無を判別できない．
(4) 構成要素自体が同定できない場合．パッケージ名や上流バージョンを特定する手がかりが得られない．静的リンクされたライブラリや，ファイル名が変更された実行ファイル等がこの段階に該当し得る．

ソースパッケージとバイナリパッケージの対応関係を保持する．同定の不確かさを単一の推定値へ集約せず，候補集合として後段の処理へ渡す．これにより，後段の判定において同定の不確かさを考慮した処理が可能となる．

上記の第 4 段階（構成要素自体が同定できない場合）では，パッケージ名及び CVE を特定できないため，VEX statement を構成する前提を欠く．この段階は VEX 判定の対象外であり，後述の判定規則は適用されない．同定不能の記録と再同定の手順は，判定対象に含まれる候補の `under_investigation` とは独立して管理する．

4.3 根拠の役割と優先順位

判定に用いる根拠の種別と適用条件を表 2 に示す．

Canonical VEX，OVAL，USN 及び changelog を主要な配布元情報とする．これらは Canonical がリリースとパッケージの組に対して公式に提供する情報であり，判定の主要な根拠となる．主要根拠間の関係として，VEX は影響状態の集約された見解を提供し，OVAL は版条件に基づく機械的な検査を可能とし，USN は修正の公開時点と対象パッケージを特定し，changelog は修正の適用履歴を配布版リビジョン単位で記録する．これらは相互に補完的であり，複数を組み合わせることで判定の確度を高められる．

バイナリパッチ存在検査は，CVE 固有の局所的な修正特徴を対象とする補助根拠として位置付ける [17]．検証済みのシグネチャにより，配布元情報から得られた判定との整合確認に用いる．配布元情報が利用できない構成要素に対しても，バイナリ検査により修正の痕跡を確認できる場合がある．ただし，バイナリ検査の結果は特定の CVE の修正特徴に限定された局所的な判断であり，配布元情報のような包括的な判定根拠とはならない．

バイナリ全体の類似度，修正特徴の未検出，及び指定シナリオでの未観測は，単独の判定根拠としない．バイナリ全体の類似度は修正箇所以外の一致を反映するに過ぎず，

© 2026 Information Processing Society of Japan

表 2: 根拠の種別と適用条件

根拠	種別	備考
Canonical VEX	主要	リリース及びパッケージの対応付けが必要
OVAL	主要	同上
USN	主要	CVE とパッケージの特定が必要
changelog	主要	版と修正の対応が明示されること
パッチ検査	補助	CVE 固有シグネチャが検証済みの場合
全体類似度	—	単独では判定根拠としない
利用観測	—	単独では判定根拠としない

特定の CVE の修正状態を示さない．修正特徴の未検出は，関数のインライン化，対象コードの独自改変又はバックポート時のパッチ調整による特徴の消失と，修正未適用を区別できない．指定シナリオでの未観測は，当該シナリオで対象コンポーネントの実行を確認できなかったことを示すにとどまり，脆弱性の不在を意味しない [15]．

4.4 VEX 状態の決定規則

本稿では VEX の 4 状態を OpenVEX 仕様 [19] に準拠して定義する．各状態に付随する記述要素として，`affected` には action statement（対応措置の記述），`not_affected` には justification（影響を受けない理由の分類）又は impact statement（理由の詳細記述），`under_investigation` には本手法が定義する保留理由（表 4）を status notes として格納する．

VEX の 4 状態の決定規則を以下に示す．

`affected` は，対象物の構成要素が Canonical のリリース及びパッケージの版条件に合致し，かつ配布元情報において影響ありと判定されている場合に付与する．

`fixed` は，対象物の構成要素に対して配布元情報により修正済みであることが確認できる場合に付与する．検証済みの CVE 固有パッチシグネチャによるバイナリ検査結果は，配布元情報による `fixed` 判定との整合性を確認する目的で参照する．バイナリ検査単独では `fixed` を付与しない．

`not_affected` は，配布元の状態と justification を対象物へ対応付けられる場合に限り付与する．本手法による独自の判断で `not_affected` を付与することは行わない．

`under_investigation` は，上記の 3 状態のいずれの決定条件も満たさない場合に付与する．根拠の欠落，曖昧さ又は矛盾がある候補は，二値判定へ強制変換せず調査中として保留する．

表 3: 根拠間の競合解決規則

競合の場面	解決規則
複数の配布版候補が同一状態	共通の状態を採用
候補間で affected と fixed が混在	`under_investigation` とし，リビジョン特定を次工程とする
主要根拠間の不一致	`under_investigation` とし，不一致箇所を記録
配布元情報とバイナリ検査の不一致	配布元情報を優先し，不一致を記録
バイナリ検査のみで fixed の根拠がある	`under_investigation`（単独では付与しない）

表 4: `under_investigation` の保留理由と次の調査工程

保留理由	次の調査工程
配布版リビジョンの対応付け不能	候補ソース，changelog 又は USN による再照合
配布元項目の不在	配布元の公開待ち又は人手調査
根拠間の競合	表 3 に従い優先根拠を決定
バイナリ検査不能	パッチソースの取得及び人手解析

注：構成要素の同定不能は VEX 判定の対象外として別管理する．

根拠間の競合が生じる場合の解決規則を表 3 に示す．

4.5 保留理由と再評価

`under_investigation` の保留理由と次の調査工程の対応を表 4 に示す．

各保留理由に対して，再同定，配布元確認，パッチ取得又は人手解析等の次工程を対応付ける．情報源の更新，対象物の変更又は判定規則の改訂を再評価契機とする．再評価の際は，保存された来歴と照合して変化箇所のみを対象とすることで，全候補の再判定を回避する．

5. 評価計画

5.1 研究課題と比較条件

本手法の有効性を検証するため，以下の研究課題を設定する．

RQ1 パッケージ管理情報の欠落によって，構成要素の同定結果と候補集合がどのように変化するか．

RQ1 は，本手法が対象とする問題の規模を定量化する．パッケージ管理情報の有無による候補集合の差を明らかにすることで，配布版情報を欠く環境における判定の必要性を実証する．

RQ2 本手法の判定規則により，どの条件で VEX 状態を確定でき，何が保留要因となるか．

RQ2 は，本手法の判定規則がどの程度の候補に対して状

© 2026 Information Processing Society of Japan

態を確定できるかを明らかにする．確定できない候補の保留理由の分布は，判定規則の改善方向を示す．

RQ3 局所的なパッチ存在検査を，どの範囲で補助根拠として利用できるか．

RQ3 は，補助根拠としてのバイナリ検査の適用可能範囲を明らかにする．配布元情報との整合件数と不整合件数の比率は，バイナリ検査の信頼性の指標となる．

比較条件として以下の 4 条件を設ける．

(1) 完全条件：パッケージ管理情報を含むすべての入力が利用可能である．
(2) 段階的除去条件：パッケージ管理情報を段階的に除去し，同定結果及び候補集合の変化を観測する．
(3) 分離条件：構成要素名と上流バージョンを固定し，配布版情報のみを除くことで，配布版情報の影響を分離する．
(4) 基準条件：判定規則を適用せず，全候補を状態の区別なく同一優先度の要調査事項として扱う．本手法の層別化による差を評価するための対照条件とする．

5.2 評価指標

各研究課題に対応する評価指標を以下に示す．

RQ1 に対しては，構成要素名，上流バージョン，配布版リビジョン及びパッケージ識別子の同定状況を条件間で比較する．候補集合については，条件間の共通，追加及び欠落件数を集計する．

RQ2 に対しては，VEX 状態別の件数（`affected`, `fixed`, `not_affected`, `under_investigation`），justification 別の件数及び保留理由別の件数を集計する．基準条件との差分により，判定規則による層別化の程度を定量化する．

RQ3 に対しては，バイナリ検査の適用可能件数，配布元情報との整合件数，不整合件数及び検査不能件数を集計する．

5.3 再現性と妥当性

対象物として，Ubuntu 公式のコンテナイメージ及び ISO イメージを用いる．公式イメージはハッシュ値が公開されており，第三者による再現が可能である．

再現性を確保するため，対象物のハッシュ値，ツールの版，データベースの取得時点，正規化規則及び判定規則の版を保存する．配布元情報は時間の経過とともに更新されるため，取得時点の情報を保存して判定の再現を可能とする．

妥当性について以下の制約を設ける．Canonical の配布元情報を入力と正解の双方に使った精度評価は行わない．配布元情報は判定の入力であり，同一の情報を正解ラベルとして用いると循環評価となるためである．独立した正解ラベルが存在しないため，Precision，Recall 及び偽陽性率等の指標は使用しない．代わりに，VEX 状態の確定比率，保留理由の分布及びバイナリ検査の適用可能範囲を評価する．これらの指標は，本手法がどの程度の候補に対して判定を完了でき，何が判定の障壁となるかを明らかにする．

6. 考察及び制約

6.1 期待される管理上の効果

本手法は，脆弱性候補を削除するのではなく，要対応（`affected`），修正済み（`fixed`），影響なし（`not_affected`）及び調査中（`under_investigation`）へ層別化する．根拠と次工程を明示することで，検収時の判定結果を保守時にも再利用可能とすることを目指す．

候補の層別化により，全候補を状態の区別なく同一優先度の要調査事項とする状況からの改善が期待される．ただし，この効果は期待効果であり，本稿の時点では達成済みとは主張しない．効率化とアラート疲労の緩和については，実運用上の工数及び認知負荷の評価を含む今後の課題とする．

6.2 適用限界

本手法の適用限界を以下に示す．

第一に，Ubuntu と Canonical の公開情報への依存がある．Canonical が情報を公開していない CVE やパッケージについては判定を行えない．Canonical の VEX データは，サポート対象のリリースに限定されるため，サポート期間を終了したリリースについては情報が得られない場合がある．

第二に，第三者リポジトリ由来のパッケージ，独自パッチを適用したビルド及びパッケージ管理情報が完全に欠落した対象物への適用は限定的である．これらの場合，配布元情報との対応付けが成立しないため，候補は `under_investigation` に留まるか，同定不能として VEX 判定の対象外となる．

第三に，バイナリパッチ存在検査において，関数のインライン化，対象コードの独自改変及びバックポート時のパッチ調整が検査精度に影響する．特に，バックポートでは上流の修正がそのまま適用されるとは限らず，周辺コードの違いに応じた改変が加えられることがある．

第四に，VEX 状態は特定の CVE に対する修正状態又は影響状態を示すものであり，悪用可能性全体を証明するものではない．`not_affected` と判定された候補であっても，justification の根拠が将来の解析により覆される可能性は排除できない．

第五に，実運用上の工数及び認知負荷は未評価である．

6.3 他ディストリビューションへの拡張

本手法は Ubuntu と Canonical の配布元情報に特化しているが，枠組み自体は他のディストリビューションにも適用可能な構造を持つ．Red Hat Enterprise Linux におけ

© 2026 Information Processing Society of Japan

る Red Hat Security Data や，Debian における Debian Security Tracker 等，主要なディストリビューションはそれぞれ固有の配布元情報を公開している．判定単位における「Ubuntu リリース」を対象ディストリビューションのリリースに置換し，配布元情報の取得先を変更することで，同様の枠組みが適用できると考えられる．ただし，配布元情報の粒度，公開形式及び更新頻度はディストリビューションごとに異なるため，根拠の対応付け規則と競合解決規則の再定義が必要となる．この拡張は今後の課題とする．

7. おわりに

本稿では，脆弱性候補の影響有無の判定とその管理の効率化に向けた基礎検討として，以下の 4 点を提示した．第一に，配布版固有情報の欠落が構成要素の同定結果と候補集合へ伝播する過程を整理した．第二に，OpenVEX 仕様に準拠した VEX 状態の判定基準と，根拠間の競合解決規則を定義した．第三に，保留理由と後続調査を対応付ける管理モデルを提示した．第四に，4 条件の比較による評価計画を設計した．

Canonical の配布元情報を主要根拠とし，バイナリパッチ存在検査を補助根拠として位置付けることで，根拠の欠落や曖昧さがある候補を調査中として保留する枠組みを示した．本稿は評価可能な枠組みの提示までであり，定量的な有効性検証を今後の課題とする．今後は，公式コンテナイメージを対象とした評価実験を実施し，VEX 状態の確定比率と保留理由の分布を明らかにする予定である．

謝辞

本研究は，JST 経済安全保障重要技術育成プログラム JPMJKP24K1 の支援により実施したものです．

参考文献

[1] 経済産業省，“ソフトウェア管理に向けた SBOM（Software Bill of Materials）の導入に関する手引 ver 2.0，” https://www.meti.go.jp/files/900016245.pdf，Accessed：2026-08-21.

[2] Ubuntu Security Team, “SecurityTeam/FAQ，” https://wiki.ubuntu.com/SecurityTeam/FAQ，Accessed：2026-08-21.

[3] Debian Project, “Debian security FAQ，” https://www.debian.org/security/faq.en.html, Accessed：2026-08-21.

[4] Executive Office of the President, “Improving the Nation’s Cybersecurity，” https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity，Accessed：2026-08-21.

[5] T. Stalnaker, N. Wintersgill, O. Chaparro, M. Di Penta, D. M. German, and D. Poshyvanyk, “BOMs Away! Inside the Minds of Stakeholders: A Comprehensive Study of Bills of Materials for Software Systems，” in Proc. 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)，2024.

[6] B. Kaur, M. Dugré, A. Hanna, and T. Glatard, “An analysis of security vulnerabilities in container images for scientific data analysis，” *GigaScience*, vol. 10, no. 6, pp. 1–7，Jun. 2021.

[7] Anchore, “False Positives and False Negatives in Vulnerability Scanning: Lessons from the Trenches，” https://anchore.com/blog/false-positives-and-false-negatives-in-vulnerability-scanning/，Accessed：2026-08-21.

[8] Canonical Ltd., “debian/ directory，” https://ubuntu.com/project/docs/how-ubuntu-is-made/concepts/debian-directory/，Accessed：2026-08-21.

[9] Canonical Ltd., “Patches，” https://ubuntu.com/project/docs/how-ubuntu-is-made/concepts/patches/，Accessed：2026-08-21.

[10] Ubuntu, “Change log for openssl package in Ubuntu，” https://launchpad.net/ubuntu/+source/openssl/+changelog，Accessed：2026-08-21.

[11] Canonical Ltd., “How to get the source of a package，” https://ubuntu.com/project/docs/contributors/setup/get-the-source-of-a-package/，Accessed：2026-08-21.

[12] Canonical Ltd., “Ubuntu VEX data，” https://documentation.ubuntu.com/security/security-updates/vex/，Accessed：2026-08-21.

[13] Canonical Ltd., “Ubuntu OVAL data，” https://documentation.ubuntu.com/security/security-updates/oval/，Accessed：2026-08-21.

[14] OpenSCAP Project, “OpenSCAP Base，” https://www.open-scap.org/tools/openscap-base/，Accessed：2026-08-21.

[15] 小津 喬, 山口 真司, 横山 浩之, “Linux システムのブートプロセス解析による脆弱性検知の誤検知低減手法，” 2026 年 暗号と情報セキュリティシンポジウム（SCIS2026），5G1-3，2026.

[16] R. Shourya, Y. Kumagai, A. C, H. Yamazaki, and H. Nakakoji, “Proposal of Vulnerability Assessment Tool for Software Supply Chain Security，” *Journal of Information Processing*, vol. 31, pp. 842–850，Dec. 2023.

[17] H. Zhang and Z. Qian, “Precise and Accurate Patch Presence Test for Binaries，” in Proc. 27th USENIX Security Symposium (USENIX Security 18)，2018.

[18] P. Deng, L. Zhang, Y. Meng, Z. Yang, Y. Zhang, and M. Yang, “ChainFuzz: Exploiting Upstream Vulnerabilities in Open-Source Supply Chains，” in Proc. 34th USENIX Security Symposium (USENIX Security 25)，2025.

[19] OpenVEX, “OpenVEX Specification v0.2.0，” https://github.com/openvex/spec/blob/v0.2.0/OPENVEX-SPEC.md，Accessed：2026-08-21.

© 2026 Information Processing Society of Japan